# Plotting Figures [1]

Many of the projects in this course require you to present (and interpret) data obtained from your computations in graphical form. There are many different plotting packages available, some commercial, some free (Yay!).

Given that most students are familiar with MATLAB and that this software is installed on the PCs in the university's computing clusters, we recommend using it, or its open source equivalent, Octave. There are also numerous other packages like gnuplot or python's matplotlib, and there are a lot of useful guides/tutorials on the internet – google them! Here are some links:

- MATLAB, is installed on all PCs in the computer cluster and is also available to all University of Manchester students for free. You can download it (to install on your own computer) from here

  `https://uk.mathworks.com/academia/tah-portal/university-of-manchester-31483744.html`

  by signing in with your university email.

- Octave (an open source MATLAB replacement) is available here

  `https://octave.org/`

- Python has great plotting capabilities too and there are numerous tutorials for the matplotlib library, including this one:

  `https://matplotlib.org/stable/tutorials/pyplot.html`

- Gnuplot is available here

  `http://www.gnuplot.info/`

  and is the lecturers' favourite! To use it in the Alan Turing computer cluster, you will have to access it via the AppsAnywhere machinery. This starts a GUI based version (a bit strange since it's really supposed to be driven from the command line) and it's also a bit odd to use. We therefore recommend using it from the Windows Command Prompt (consult the `compiling.pdf` document discussed in the first lab session). Unfortunately, this currently requires a slightly tedious start-up procedure.

  1. Shut down your existing Command Prompt windows (if any).
  2. Start gnuplot from AppsAnywhere.
  3. Shut it down again (unless you do want to use the GUI version after all!).
  4. Reopen a Command Prompt window.
  5. Navigate to the directory that contains your data (using `cd`).
  6. Start gnuplot, by typing `gnuplot`.

[1] Version 2026/09/24.
Feedback, comments, corrections, applause, etc. to: `M.Heil@maths.manchester.ac.uk`



2026/09/24, 09:49:00

7. Once you're on the gnuplot command line you can interactively enter any plotting commands or load scripts such as the ones provided on the course's Canvas page:

```
# This plots a sin curve
plot sin(x)

# This reads in a script (saves you having to type
# the commands over and over again)
load "fibonacci_project_basic.gp"
```

   Type `help` to get, well, help, and `quit` to, hmmm, quit. See how easy it is?

- Finally, paraview is an extremely powerful open source package for the 3D plotting. You won't need it for any of the projects in this course, but I thought I'd mention it anyway:

  https://www.paraview.org/

# Plotting in MATLAB or Octave

Here we provide a brief introduction explaining how to take data stored in a plain text file (that may have been generated by your C++ program), generate simple line and scatter plots, and export it in postscript (`.eps`) or pdf format.

## Loading data

It is very straightforward to read text files containing a matrix of double-precision numerical values in MATLAB/Octave. We will illustrate this with an example data file `data.dat` (which may have been output from a C++ program), containing the four lines,

```
-1.0 1.0
-1.5 2.25
-2.0 4.0
-2.5 6.25
```

The command

```
data = load('data.dat');
```

creates a matrix called `data` with 2 columns and 4 rows, containing the data from the text file.

It is important that every line in the file contains the same number of numerical values. There must also be no other data in the file (for example no text or blank lines). However, the formatting of the matrix of floating-point values is quite flexible: the numbers can be separated by spaces (as above), by tabs or by commas. You can also use the `readmatrix` function in place of `load`.

## Plotting data

Having loaded the data, as above, we usually need to select particular values, or particular rows or columns for plotting. The syntax data(i,j) selects the value at the ith row and the jth



2026/09/24, 09:49:00

column from a matrix. Note that, unlike C++, indexes in MATLAB/Octave start at 1, and so `data(1,1)` is the top-left element in the data (`-1.0`, in the data above). The MATLAB colon operator `(:)` is used to select all rows or columns from a matrix. For example `data(1,:)` returns the elements in row 1 and in any column – i.e. the row-vector corresponding to the first row of the matrix (-1.0, 1.0, in the data above). The expression `data(:,1)` is a column-vector corresponding to the first column of the matrix `(-1.0, -1.5, -2.0, -2.5)`. Simple line or scatter plots are produced in MATLAB/Octave with the plot command, which usually takes two arguments, for the x and y data values. For example,

```
plot(data(:,1), data(:,2));
```

plots values in the first column of data against corresponding values in the second column. With the data loaded as above, the (x, y) coordinates of the four points plotted by this command are therefore $(-1, 1), (-1.5, 2.25), (-2, 4)$ and $(-2.5, 6.25)$.

By default, plot plots lines between these data points. A third, optional, argument specifies the plot style. The examples below demonstrate how some different plot styles are achieved.

```
plot(data(:,1), data(:,2), '-'); % line plot (default)
plot(data(:,1), data(:,2), 'x'); % scatter plot with crosses
plot(data(:,1), data(:,2), '.'); % scatter plot with dots
plot(data(:,1), data(:,2), '-*'); % line plot with stars at data
plot(data(:,1), data(:,2), '-o'); % line plot with circles at data
plot(data(:,1), data(:,2), '--'); % dashed line plot
plot(data(:,1), data(:,2), '-.'); % dash-dotted line plot
plot(data(:,1), data(:,2), 'b'); % blue plot
plot(data(:,1), data(:,2), 'r'); % red plot
plot(data(:,1), data(:,2), 'k.'); % scatter plot with black dots
plot(data(:,1), data(:,2), 'g--'); % plot with green dashed lines
```

It is possible to specify a wider range of colours using RGB (red/green/blue) values that go from 0 to 1,

```
plot(data(:,1), data(:,2), '--', 'Color', [1, 0.5, 0]); % plot with
    orange dashed lines
```

and the line width with

```
plot(data(:,1), data(:,2), 'LineWidth', 1); % plot with line width 1
```

The default line width is 0.5. More options for changing the appearance of a plot are given in the MATLAB documentation (`http://uk.mathworks.com/help/matlab/ref/plot.html`).

Note that, by default, a `plot` command will clear the results of any plotting that has been done previously. In order to plot multiple curves, use the command

```
hold on;
```

after the first `plot` command. Use `hold off` to return MATLAB's plotting to the original behaviour.

Almost all plots require labels on the axes. In MATLAB/Octave this is achieved with the commands

```
xlabel('x label'); ylabel('y label');
```



2026/09/24, 09:49:00

For figures using multiple plot commands, legends can be produced as follows:

```
% Make three plots of different styles on one axis:
plot(data(1,1), data(1,2), 'bx');
hold on; % remember we need "hold on" when using multiple plots
plot(data(2,1), data(2,2), 'ro');
plot(data(3,1), data(3,2), 'g*');
legend('Row 1','Row 2','Row3','Location','NorthOutside');
```

The first three arguments of the `legend` command relate to each of the corresponding three `plot` commands. The last two arguments `'Location', 'NorthOutside'` display the legend above the plot; see the MATLAB documentation (`http://uk.mathworks.com/help/matlab/ref/legend.html`) for more options.

## Exporting plots

To save a plot to an encapsulated postscript file that can be imported into LaTeX, use the commands

```
set(gcf, 'PaperPosition', [0 0 8 6]);
print('-depsc','example_figure.eps');
```

The first of these commands sets the size of the exported figure (here, to 8cm × 6cm, a reasonable size for a figure in an A4 document). The second writes the figure to an EPS (encapsulated postscript) file. This filetype enables the use of figures that can be zoomed in on without any pixelation (unlike image filetypes such as `.png`).

If you use the beamer style that I use for the model solutions for the projects, you won't be able to use eps files. To create pdf files without lots of whitespace padding around the actual figure, use the following commands:

```
set(gcf,'PaperOrientation','landscape');
set(gcf,'PaperUnits','normalized');
set(gcf,'PaperPosition', [0 0 1 1]);
print(gcf, '-dpdf', 'example_figure.pdf');
```

This pdf figure can then be incorporated into a LaTeX document with

```
\includegraphics[width=10cm]{example_figure.pdf}
```



2026/09/24, 09:49:00